



Unsupervised Machine Learning

A practical guide to **Unsupervised Machine Learning**, covering clustering, dimensionality reduction, important algorithms, evaluation techniques, and Python implementation.



What is Unsupervised Machine Learning?

Unsupervised Machine Learning is a branch of Machine Learning where the model learns patterns, relationships, and structures from data **without using predefined target labels**.

In supervised learning, we provide:

Input Data + Target → Model → Prediction

In unsupervised learning:

Input Data → Model → Hidden Patterns / Groups

Example

Suppose we have customer data:

Age	Annual Income	Spending Score
22	25,000	80
25	28,000	75
45	90,000	20
50	95,000	15

There is no column telling us which customers are "budget", "regular", or "premium".

An unsupervised learning algorithm can discover these groups automatically.

Objectives

In this project, we will learn:

- What Unsupervised Learning is
 - Difference between supervised and unsupervised learning
 - Clustering
 - K-Means Clustering
 - Elbow Method
 - Silhouette Score
 - Hierarchical Clustering
 - DBSCAN
 - Dimensionality Reduction
 - PCA
 - Data preprocessing
 - Feature scaling
 - Visualization
 - Real-world applications
-

Types of Unsupervised Learning

The two major categories are:

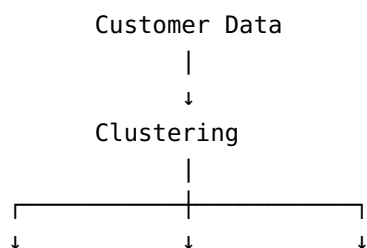
1. Clustering

Clustering divides data into groups based on similarity.

Popular algorithms:

- K-Means
- Hierarchical Clustering
- DBSCAN
- Gaussian Mixture Models

Example:



Cluster 1	Cluster 2	Cluster 3
Budget	Regular	Premium

2. Dimensionality Reduction

Dimensionality reduction reduces the number of features while preserving important information.

Popular techniques:

- PCA
- t-SNE
- UMAP

Example:

```
100 Features
  ↓
  PCA
  ↓
2 Principal Components
```

It can be useful for:

- Data visualization
- Noise reduction
- Faster computation
- Feature compression

K-Means Clustering

K-Means is one of the most popular clustering algorithms.

The goal is to divide observations into clusters.

How K-Means Works

Step 1 — Choose K

Select the number of clusters.

For example:

K = 3

Step 2 — Initialize Centroids

The algorithm selects initial cluster centers.

Step 3 — Assign Data Points

Each observation is assigned to the nearest centroid.

Step 4 — Recalculate Centroids

The centroid of each cluster is recalculated.

Step 5 — Repeat

The process continues until the centroids stabilize.

K-Means Implementation

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Load dataset
df = pd.read_csv("customer_data.csv")

# Select features
X = df[["Annual Income", "Spending Score"]]

# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Create K-Means model
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
```

```
# Fit model and create clusters
df["Cluster"] = kmeans.fit_predict(X_scaled)

print(df.head())
```



Why Feature Scaling?

K-Means uses distance to determine similarity.

If one feature has a much larger numerical range than another feature, it can dominate the distance calculation.

For example:

Age	→ 18 - 70
Annual Income	→ 20,000 - 150,000

Annual Income has a much larger scale.

Therefore, scaling is usually important before applying distance-based algorithms.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```



Elbow Method

One of the challenges of K-Means is determining the optimal value of **K**.

The **Elbow Method** can help.

We calculate the model's inertia for different values of K.

```
inertia = []

for k in range(1, 11):
```

```

model = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

model.fit(X_scaled)

inertia.append(model.inertia_)

```

Plot the results:

```

plt.plot(
    range(1, 11),
    inertia,
    marker="o"
)

plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")

plt.show()

```

The point where the decrease in inertia starts slowing down is commonly called the **elbow**.

That value can be a reasonable starting point for choosing K.



Visualizing K-Means Clusters

```

plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=df["Cluster"]
)

plt.xlabel("Annual Income")
plt.ylabel("Spending Score")

plt.title("Customer Segmentation using K-Means")

```

```
plt.show()
```

This visualization helps us understand how the algorithm divided the customers.

Silhouette Score

The Silhouette Score measures how well-separated the clusters are.

It compares:

- How close a point is to its own cluster
- How far it is from other clusters

```
from sklearn.metrics import silhouette_score

score = silhouette_score(
    X_scaled,
    df["Cluster"]
)

print("Silhouette Score:", score)
```

The score ranges from:

-1 → 1

Generally:

- Higher score → better-defined clusters
- Around 0 → overlapping clusters
- Negative score → possible incorrect assignments

The Silhouette Score should be interpreted together with visualization and domain knowledge rather than used as the only decision criterion.

Hierarchical Clustering

Hierarchical Clustering creates a hierarchy of clusters.

It can be visualized using a **dendrogram**.

There are two common approaches:

Agglomerative

Starts with individual observations and repeatedly merges the closest groups.

Divisive

Starts with all observations in one group and repeatedly splits them.



Hierarchical Clustering Implementation

```
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

linked = linkage(
    X_scaled,
    method="ward"
)

plt.figure(figsize=(10, 6))

dendrogram(linked)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")

plt.show()
```

The dendrogram can help us understand the hierarchical structure of the data and choose a possible number of clusters.



DBSCAN

DBSCAN stands for:

Density-Based Spatial Clustering of Applications with Noise

Unlike K-Means, DBSCAN is based on the density of observations.

It can identify:

- Dense regions
- Sparse regions
- Noise/outliers

Example:

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(
    eps=0.5,
    min_samples=5
)

df["DBSCAN_Cluster"] = dbscan.fit_predict(X_scaled)

print(df.head())
```

In DBSCAN, the label `-1` commonly represents points identified as noise.

PCA

PCA stands for:

Principal Component Analysis

PCA is a dimensionality-reduction technique.

Suppose we have:

```
20 Features
  ↓
  PCA
  ↓
 2 Principal Components
```

Instead of working with all 20 original features, we can represent much of the variation in the data using fewer components.



PCA Implementation

```
from sklearn.decomposition import PCA

pca = PCA(
    n_components=2
)

X_pca = pca.fit_transform(X_scaled)

print(X_pca.shape)
```

Visualize the transformed data:

```
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1]
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.title("PCA Visualization")

plt.show()
```



Explained Variance

PCA provides information about how much variance each principal component explains.

```
print(pca.explained_variance_ratio_)
```

We can also calculate the cumulative explained variance:

```
print(
    pca.explained_variance_ratio_.cumsum()
)
```

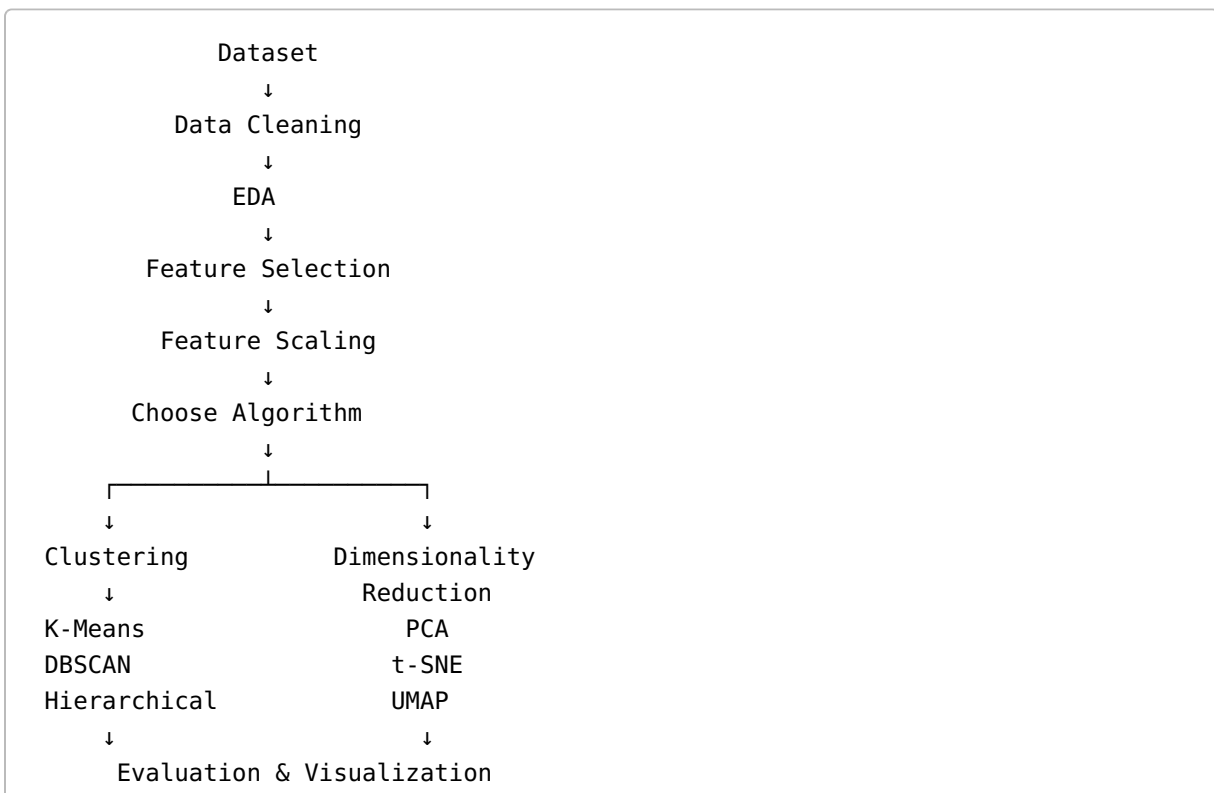
This helps us understand how much information is retained after dimensionality reduction.

Supervised vs Unsupervised Learning

Feature	Supervised Learning	Unsupervised Learning
Target Variable	Required	Not required
Labels	Available	Usually unavailable
Goal	Prediction	Pattern Discovery
Classification	Yes	No
Regression	Yes	No
Clustering	No	Yes
PCA	No	Yes
Example	Spam Detection	Customer Segmentation

Unsupervised ML Workflow

A typical workflow looks like this:



↓
Final Insights

Important Preprocessing Steps

Before applying unsupervised algorithms, we should inspect the dataset carefully.

1. Handle Missing Values

```
df.isnull().sum()
```

Possible approaches:

```
df.dropna()
```

or:

```
df.fillna(df.mean(numeric_only=True))
```

2. Remove Duplicates

```
df.drop_duplicates(inplace=True)
```

3. Select Relevant Features

```
X = df[
    [
        "Age",
        "Annual Income",
        "Spending Score"
    ]
]
```

4. Scale Features

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```



Distance Metrics

Many clustering algorithms depend on measuring similarity or distance.

Common distance metrics include:

- Euclidean Distance
- Manhattan Distance
- Cosine Distance

The choice of distance metric can affect clustering results.



Evaluation Techniques

Since unsupervised learning usually doesn't have known target labels, evaluation is different from supervised learning.

Common techniques include:

Internal Evaluation

- Silhouette Score
- Davies-Bouldin Index
- Calinski-Harabasz Index
- K-Means inertia

Visual Evaluation

- Scatter plots
- PCA plots
- Dendrograms

Business/Domain Evaluation

Ask:

Do the discovered groups actually make sense in the real-world problem?

This is extremely important because a mathematically valid cluster is not automatically a useful business segment.

Real-World Applications

Unsupervised Machine Learning is used in many areas.

Customer Segmentation

Companies can group customers according to:

- Spending behavior
- Income
- Purchase frequency
- Preferences

Anomaly Detection

Identify unusual transactions or observations.

Recommendation Systems

Find users or products with similar characteristics.

Healthcare & Biology

Discover groups or patterns in biological datasets.

Market Segmentation

Identify groups of customers with similar behavior.

Image Processing

Reduce image dimensions or group similar images.

Text Analysis

Discover topics or groups of similar documents.

Common Mistakes

Using unscaled features with distance-based algorithms

Different feature ranges can distort distances.

Choosing K randomly

Use methods such as the Elbow Method and Silhouette Score as starting points.

Ignoring outliers

Outliers can significantly affect some clustering algorithms.

Assuming every cluster has business meaning

Always interpret clusters using domain knowledge.

Evaluating only with one metric

Use multiple evaluation methods where appropriate.

Mini Project: Customer Segmentation

Problem Statement

A company wants to understand its customers and divide them into meaningful groups based on their characteristics and spending behavior.

Dataset Features

CustomerID
Age
Annual Income
Spending Score

Project Objective

Use unsupervised machine learning to identify different customer segments.

Project Steps

1. Load Dataset
2. Understand Dataset
3. Clean Data
4. Perform EDA
5. Select Features
6. Scale Features
7. Find Optimal K
8. Train K-Means
9. Visualize Clusters
10. Calculate Silhouette Score
11. Interpret Customer Segments
12. Generate Business Insights

Recommended Project Structure

```
unsupervised-ml/  
|  
├── README.md  
|  
├── data/  
│   └── customer_data.csv  
|  
├── notebooks/  
│   └── customer_segmentation.ipynb  
|  
├── images/  
│   ├── elbow_curve.png  
│   ├── clusters.png  
│   └── pca.png  
|  
├── requirements.txt  
|  
└── LICENSE
```

Requirements

Create a `requirements.txt` file:

```
pandas  
numpy  
matplotlib  
scikit-learn  
scipy
```

Install dependencies:

```
pip install -r requirements.txt
```

Key Takeaways

After completing this project, you should understand:

- What Unsupervised Machine Learning is
- How clustering works
- How K-Means works
- How to select K using the Elbow Method
- How to evaluate clusters using Silhouette Score
- How Hierarchical Clustering works
- How DBSCAN identifies dense regions and noise
- How PCA reduces dimensionality
- Why feature scaling matters
- How unsupervised learning is applied to real-world problems

Future Improvements

Possible extensions of this project:

- Add DBSCAN comparison
 - Add Hierarchical Clustering comparison
 - Perform PCA before clustering
 - Compare different distance metrics
 - Experiment with different datasets
 - Build an interactive dashboard
 - Deploy the model using Streamlit
 - Add automated cluster profiling
-



Author

Anushka Tyagi

B.Tech Computer Science & Engineering
Machine Learning | Data Analytics | Python

★ If you found this repository useful

Give the repository a ★ and feel free to explore the code and experiment with different datasets.

**MachineLearning #UnsupervisedLearning
#Python #DataScience #KMeans #Clustering #PCA
#DBSCAN #ArtificialIntelligence**